

Manual de Engenharia e Arquitetura de Software

Portal Gerencial de Indicadores — Farmácias Conviva

Guia Técnico & Estudo de Caso de Engenharia:

Este documento apresenta a arquitetura detalhada, decisões de design, padrões de projeto (Design Patterns), estrutura de dados em nuvem e estratégias de alta performance implementadas no Portal de Acompanhamento Semanal e Gestão de Demandas.



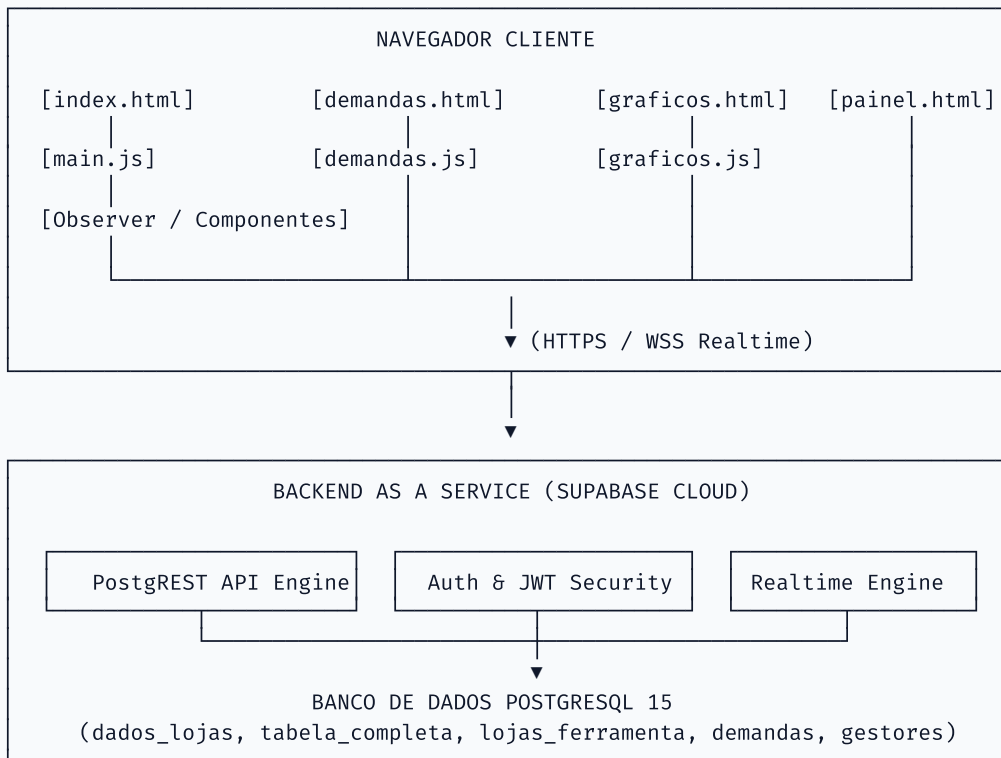
Sumário Executivo

Seção	Tópico	Conceitos de Engenharia Abordados
1	Visão Geral & Estilo Arquitetural	JAMstack, Serverless BaaS, MPA vs SPA, Desempenho
2	Árvore de Diretórios e Componentização	Separação de Responsabilidades (SoC), Clean Architecture
3	Padrões de Projeto (Design Patterns)	Observer Pattern, Singleton, Adapter/Presenter, RBAC
4	Modelagem de Dados & PostgreSQL	Relacionamentos, Índices, RLS Security, Free Tier Optimization
5	Motor de Diagnóstico & Regras de Negócio	Avaliação Multicritério, Comparador Histórico, Transições
6	Qualidade, Segurança e Confiabilidade	Cache-Busting, Defesa contra XSS, Tolerância a Falhas

💡 **Objetivo Pedagógico:** Este material foi estruturado para servir de referência acadêmica e profissional, demonstrando como princípios teóricos de Engenharia de Software (SOLID, DRY, Clean Code) são aplicados em um software corporativo real.

1. Visão Geral e Estilo Arquitetural

O Portal de Indicadores foi construído sob o paradigma moderno da **JAMstack** (**JavaScript, APIs, Markup**), combinado com uma arquitetura **Multi-Page Application (MPA) Componentizada**.



Por que não usar um Framework Pesado (ex: React, Angular)?

Em projetos de inteligência operacional com tabelas densas e leitura de alta frequência, a escolha de **JavaScript Vanilla ES6+** oferece vantagens críticas:

- **Desempenho Extremo (Zero Overhead):** Sem Virtual DOM consumindo CPU/RAM, o navegador renderiza as matrizes de dados diretamente no DOM nativo.
- **Carregamento Instantâneo:** O tamanho total dos arquivos transferidos é inferior a 150 KB (contra mais de 1.5 MB de bundles React/Angular).
- **Longevidade e Manutenibilidade:** Sem dependências que sofrem depreciação constante de versões (zero risco de quebra por updates de bibliotecas terceiras).

2. Árvore de Diretórios e Separação de Camadas

O projeto segue o princípio de **Separação de Responsabilidades (Separation of Concerns - SoC)**, onde cada pasta possui um papel estrito:

1. Camada de Estilos (`css/`)

`css/globais/` : Contém o Design System central (`variaveis.css` define a paleta corporativa Conviva, tipografia e sombras; `base.css` aplica reset e scrollbars; `layout.css` gerencia o grid principal).

`css/componentes/` : Estilos atômicos e isolados de cada tabela, header, filtros, diagnóstico e modais.

`css/paginas/` : Estilos específicos das rotas de entrada (painel, login, gráficos, demandas).

2. Camada de Infraestrutura e Serviços (`js/servicos/`)

`supabaseClient.js` : Ponto único de comunicação com o Supabase. Garante reuso de conexão.

`FiltroObserver.js` : Barramento de eventos que sincroniza os filtros com as tabelas.

3. Camada de Domínio e Componentes (`js/componentes/`)

`tabela-principal.js` : Monta a matriz retroativa de 5 semanas da filial.

`tabela-media.js` : Processa o cálculo cruzado de média da classe com segmentação por ferramenta (*Analysis vs Pricing*).

`diagnostico.js` : Motor de inteligência operacional que compara KPIs e gera os pontos fortes e de atenção.

`demandas.js` : Módulo CRUD com controle de acesso granular baseado em autor e perfil de gestor.

3. Padrões de Projeto (Design Patterns) na Prática

A. Observer Pattern (Padrão Observador)

O **Observer** resolve o problema de comunicação entre componentes desconectados. Quando o usuário troca a loja ou semana na barra de filtros, não chamamos cada tabela manualmente; em vez disso, o `filtroObserver` notifica todos os ouvintes inscritos:

```
// js/servicos/FiltroObserver.js
class FiltroObserver {
  constructor() {
    this.ouvintes = [];
  }

  inscrever(funcao) {
    this.ouvintes.push(funcao);
  }

  notificar(dados) {
    this.ouvintes.forEach(funcao => funcao(dados));
  }
}

export const filtroObserver = new FiltroObserver();
```

```
// main.js - Inscrição Desacoplada dos Componentes
filtroObserver.inscrever((dados) => {
  preencherTabelaPrincipal(supabase, dados.filial, dados.periodo);
});

filtroObserver.inscrever((dados) => {
  renderizarDiagnostico(supabase, dados.filial, dados.periodo);
});
```

B. Singleton Pattern (Instância Única)

O **Singleton** assegura que apenas um objeto do cliente Supabase exista durante toda a execução da aplicação, evitando o esgotamento de conexões no plano gratuito:

```
// js/servicos/supabaseClient.js
import { createClient } from 'https://cdn.jsdelivr.net/npm/@supabase/supabase-js/+esm';

const supabaseUrl = 'https://zhfisczfrmhxeegcvvto.supabase.co';
const supabaseKey = '...';

// Exporta uma única instância constante compartilhada
export const supabase = createClient(supabaseUrl, supabaseKey);
```

C. Role-Based Access Control (RBAC) em Camada de Aplicação

Em `demandas.js`, implementamos uma checagem de autorização em dois níveis (Front-end e Banco):

- **Autor da Demanda:** `demanda.analista === usuarioLogado` → Permite Editar, Concluir e Excluir.
- **Administrador Geral:** Presente na tabela `gestores` → Permissão irrestrita sobre todas as demandas.
- **Terceiros:** Exibição do badge `🔒 Somente Leitura` e bloqueio de cliques.

4. Modelagem de Dados & Otimização no PostgreSQL

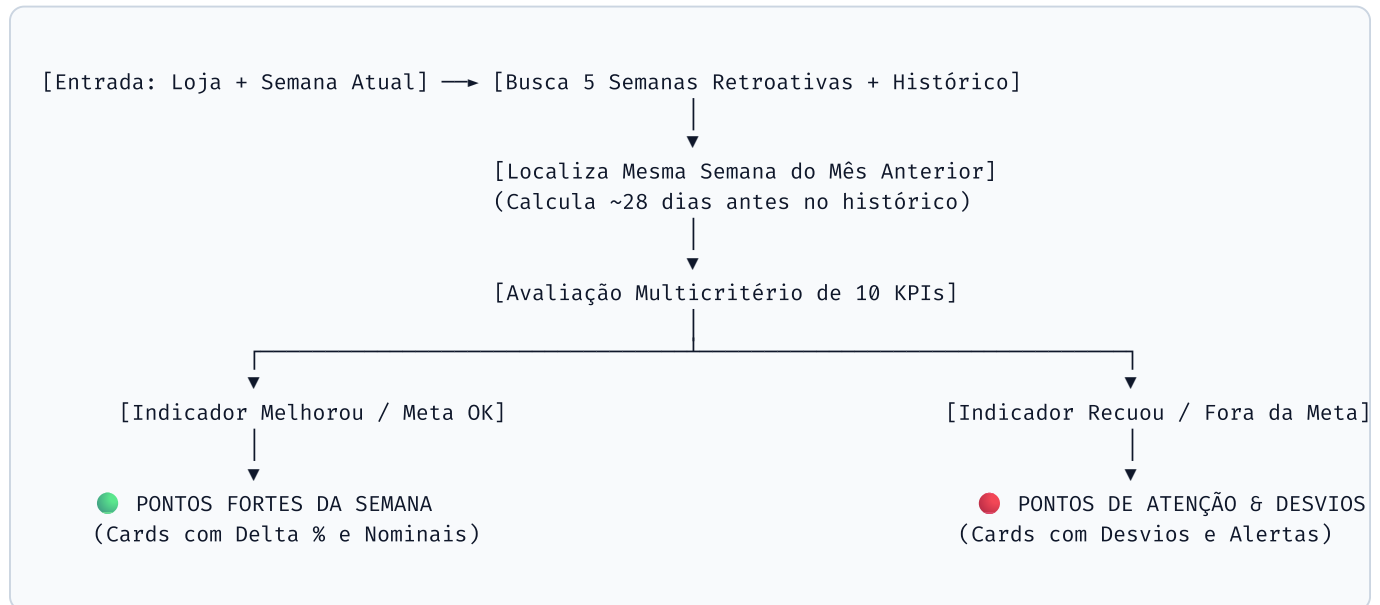
As tabelas foram modeladas para garantir integridade e leituras pontuais rápidas:

Tabela	Chaves Principais	Finalidade
<code>dados_lojas</code>	<code>filial</code> , <code>data_final</code>	Armazena os indicadores semanais (Venda Líquida, Qtd Clientes, Ticket, CMV, Falta, Desconto).
<code>lojas_ferramenta_desconto</code>	<code>loja</code>	Mapeia qual ferramenta de desconto (<code>ANALYSIS</code> ou <code>PRICING</code>) a loja opera.
<code>tabela_completa</code>	<code>filial</code> , <code>status</code> , <code>data_final</code>	Histórico geral da rede usado para calcular a média de desconto entre lojas da mesma classe e ferramenta.
<code>meta_cluster</code>	<code>cluster</code>	Metas ideais de mercado (CMV <= 63%, Desconto <= 30%, Falta <= 9%, Ticket >= R\$ 50,00).
<code>demandas</code>	<code>id</code> , <code>loja</code> , <code>analista</code>	Gestão de planos de ação, prazos, prioridade e status de conclusão.

 **Otimização Free Tier:** As consultas utilizam projeções restritas, ordenação indexada por `data_final DESC` e limites de paginação (`.limit()`), mantendo o consumo de banda mensal abaixo de 5% da cota gratuita.

5. Motor do Diagnóstico Semanal (Inteligência de Negócios)

O módulo `diagnostico.js` processa regras de negócio no próprio navegador do cliente:



Os 10 KPIs Monitorados:

1. **Venda Líquida:** Faturamento e crescimento percentual vs período comparado.
2. **Quantidade de Clientes:** Atendimentos e variação nominal de fluxo.
3. **Ticket Médio:** Gasto médio por atendimento (Meta: $\geq$ R\$ 50,00).
4. **Itens por Cliente:** Cesta média de produtos (Meta: $\geq$ 2,50).
5. **CMV (Custo da Mercadoria):** Margem e custo dos produtos (Teto: $\leq$ 63,0%).
6. **Desconto:** Concessão de desconto comercial (Teto: $\leq$ 30,0%).
7. **Falta de Produtos (Ruptura):** Nível de desabastecimento em loja (Meta: $\leq$ 9,0%).
8. **Aproveitamento Promocional:** Adesão aos encartes e ofertas (Meta: $\geq$ 60,0%).
9. **Cobertura de Estoque:** Dias de estoque disponível (Calibragem: 45 a 60 dias).
10. **EAS (Salão de Vendas):** Índice de eficiência visual e operacional (Meta: $\leq$ 20,0%).

6. Confiabilidade e Melhores Práticas de Engenharia

A. Cache-Busting Contínuo (`?v=XX`)

Navegadores realizam cache agressivo de arquivos JavaScript e CSS. Para garantir que todos os colaboradores recebam as atualizações sem precisar limpar o histórico, aplicamos versões dinâmicas em todas as importações:

```
<script type="module" src="main.js?v=85"></script>
<link rel="stylesheet" href="css/componentes/diagnostico.css?v=61">
```

B. Código Defensivo e Tolerância a Falhas

- **Optional Chaining** (`?.`): Evita erros de `TypeError: Cannot read property of undefined` em elementos ausentes.
- **Operadores de Coalescência Nula** (`|| '-'`): Garante que dados nulos do banco não quebrem as tabelas.
- **Sanitização e Conversão Numérica** (`normalizarPercentual`): Trata automaticamente números salvos como `0.25` ou `25%`.

Conclusão para o Estudante de Engenharia:

A força deste sistema reside na sua arquitetura limpa: componentes altamente coesos, desacoplados via Observer, banco relacional eficiente e uma interface que entrega alto valor de negócio sem desperdício de recursos computacionais.